

Unit 10: More on Files

CONTENTS

Objectives

Introduction

- 10.1 Concept of Streams
- 10.2 Hierarchy of Console Stream Classes
- 10.3 Reading/Writing a Character from/into a File
- 10.4 File pointer & manipulator
- 10.5 Sequential Input and Output Operations
- 10.6 Update a File
- 10.7 Different Types of Files
- 10.8 Types of File Systems
- 10.9 Binary Files
- 10.10 Command Line Argument in C++

Summary

Keywords

Self Assessment

Review Questions

Answers for Self Assessment

Further Readings

Objectives

After studying this unit, you will be able to:

- Recognize the concepts of streams
- Describe the hierarchy of console stream classes
- Explain the unformatted I/O operations
- Discuss the managing output with manipulators
- Describe the appending in a file
- Discuss the different types of files
- Discuss the command line arguments

Introduction

One of the most essential features of interactive programming is its ability to interact with the users through operator console usually comprising keyboard and monitor. Accordingly, every computer language (and compiler) provides standard input/output functions and/or methods to facilitate console operations.

C++ accomplishes input/output operations using concept of stream. A stream is a series of bytes whose value depends on the variable in which it is stored. This way, C++ is able to treat all the input and output operations in a uniform manner. Thus, whether it is reading from a file or from the keyboard, for a C++ program it is simply a stream.

Programs frequently read data from a data file and write the result to another (or the same) data file. This section will cover concerns with using a C++ programme to access data from a data file.

10.1 Concept of Streams

A stream is a source of sequence of bytes. A stream abstracts for input/output devices. It can be tied up with any I/O device and I/O can be performed in a uniform way. The C++ iostream library is an object-oriented implementation of this abstraction. It has a source (producer) of flow of bytes and a sink (consumer) of the bytes. The required classes for the stream I/O are defined in different library header files.

To use the I/O streams in a C++ program, one must include iostream.h header file in the program. This file defines the required classes and provides the buffering. Instead of functions, the library provides operators to carry out the I/O. Two of the Stream Operators are:

<< : Stream insertion for output.

>> : Stream extraction for input.

The following streams are created and opened automatically:

cin : Standard console input (keyboard).

cout : Standard console output (screen).

cprn : Standard printer (LPT1).

cerr : Standard error output (screen).

clog : Standard log (screen).

caux : Standard auxiliary (screen).



Example

```
#include <iostream>          // Header for stream I/O.

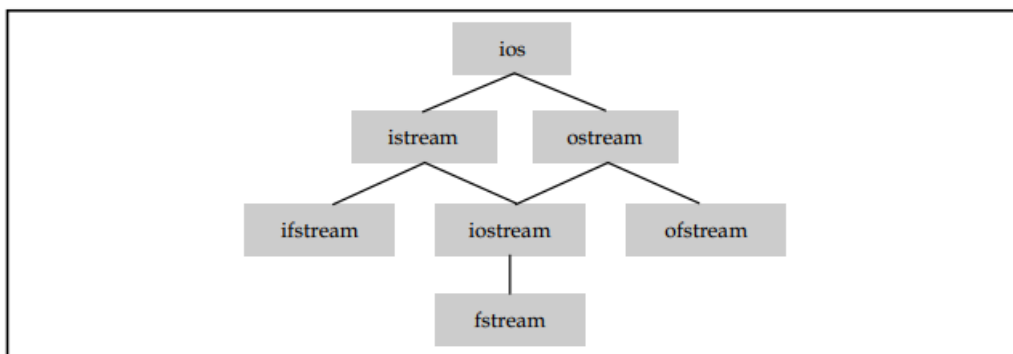
int main(void)
{
    int x;                    // variable to hold the input integer
    cout << "Enter an integer: ";
    cin >> x;
    cout << "\n You have entered " << x;
}
```

10.2 Hierarchy of Console Stream Classes

Data files can also be linked to streams. fstream.h and/or strstream.h define the essential stream classes for file I/O.

fstream : File I/O class.

ifstream	Input file class.
istrstream	Input string class.
ofstream	Output file class.
ostrstream	Output string class.
strstream	String I/O class.



There are some special functions that can alter the state the stream. These functions are called manipulators. Stream manipulators are defined in `iomanip.h`.

dec	Sets base 10 integers.
endl	Sends a new line character.
ends	Sends a null (end of string) character.
flush	Flushes an output stream.
fixed	Sets fixed real number notation.
oct	Sets base 8 integers.
ws	Discard white space on input.
setbase(int)	Sets integer conversion base (0, 8, 10 or 16 where 0 sets base 10).
setfill(int)	Sets fill character.
setprecision(int)	Sets precision.
setw(int)	Sets field width.
resetiosflags(long)	Clears format state as specified by argument.

A file may be opened for a number of file operations. The corresponding stream must be set with the intended operation. The different file stream modes are indicated by File Access Flags as listed below:

ios::app	Open in append mode.
ios::ate	Open and seek to end of file.
ios::in	Open in input mode.
ios::nocreate	Fail if file doesn't already exist.
ios::noreplace	Fail if file already exists.
ios::out	Open in output mode.
ios::trunc	Open and truncate to zero length.
ios::binary	Open as a binary stream.

10.3 Reading/Writing a Character from/into a File

Reading and writing a character in a data file has been dealt with in the previous sections in detail. The general procedure of reading a file one character at a time is listed below:

1. Create an input file stream from <fstream.h> header file:

```
ifstream name_of_input_stream;
```

2. Open the data file by passing the file name (optionally full name) to this input stream:

```
name_of_input_stream.open("data.dat");
```

Both the above statements can be combined in the following:

```
ifstream name_of_input_stream("data.dat");
```

3. Set up a character type variable to hold the read character.

```
char ch;
```

4. Read a character from the opened file using get() function:

```
name_of_input_stream.get(ch);
```

This way you can read the entire file in a loop stopping condition of the loop being the end of file:

```
while(!filename.eof())
{
    name_of_input_stream.get(ch);
    //process the read character
}
```

5. When finished close the file using close() function:

```
name_of_input_stream.close();
```

The if stream class is defined in fstream.h header file. Therefore you must include this file in your program. The complete program is listed below.

```
//reading a file one character at a time
#include <fstream.h>

void main() //the program starts here
{

    ifstream filename("c:\\cppio.dat");
    char ch;
```

```

while(!filename.eof())
{
filename.get(ch);
cout << ch;
}
filename.close();
}

```

The general procedure of writing one character at a time in a file is listed below:

1. Create an output file stream from <fstream.h> header file:

```
ofstream name_of_output_stream;
```

2. Open the data file by passing the file name (optionally full name) to this output stream:

```
name_of_output_stream.open("data.dat");
```

Both the above statements can be combined in the following:

```
ofstream name_of_output_stream("data.dat");
```

3. Write a character in the opened file using << operator:

```
name_of_output_stream << 'A';
```

4. When finished close the file using close() function:

```
name_of_output_stream.close();
```



Lab Exercise

//Program to write in file

```

#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream of("demo.txt");
    of<< "Writing to file using fstream constructor!" << endl;
    of.close ();
    return 0;
}

```



Lab Exercise

//Program to read from file

```

#include <iostream>
#include <fstream>
using namespace std;
int main()

```

```

{
string text;
ifstream ReadFile("a.txt");
while (getline (ReadFile, text)) {
    cout << text;
}
ReadFile.close();
return 0;
}

```



Note: - Create **a.text** file and write something in the file before execute above program.

10.4 File pointer & manipulator

Each file object has two integer values associated with it:

get pointer and put pointer

These values specify the byte number in the file where reading or writing will take place.

By default reading pointer is set at the beginning and writing pointer is set at the end (when file open in ios::app mode)

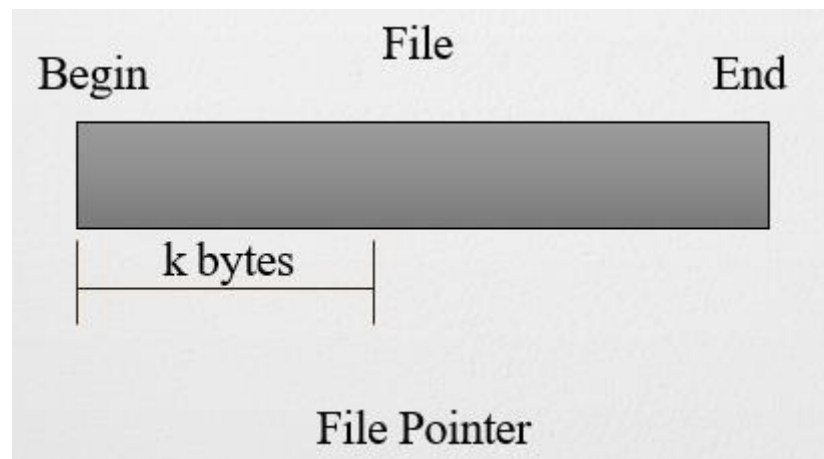
Functions associated with file pointers:

- The **seekg()** and **tellg()** functions allow you to set and examine the get pointer.
- The **seekp()** and **tellp()** functions allow you to set and examine the put pointer.

seekg() function

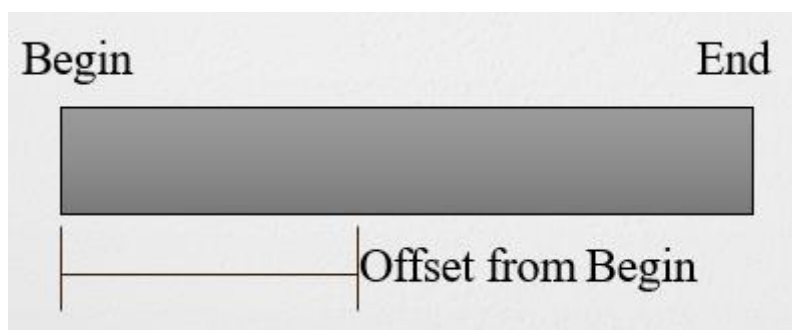
With one argument:

seekg(k) where k is absolute position from the beginning. The start of the file is byte 0.



With two argument:

- The first argument represents an offset from a particular location in the file.
- The second specifies the location from which the offset is measured.

***tellg() function***

- The tellg() function is used with input streams and returns the current “get” position of the pointer in the stream.

**Lab Exercise**

//Program

#include <iostream>

#include <fstream>

using namespace std;

int main()

{

fstream file;

file.open("details.txt",ios::out);

if(!file){

cout<<"Error";

}

file<<"OOP using C++";

cout<<"current position in file"<<file.tellp()<<endl;

file.close();

file.open("details.txt",ios::in);

if(!file){

cout<<"error";

}

cout<<"After open again file position is "<<file.tellg()<<endl;

char ch;

```

while(!file.eof()){
    cout<<"Position"<<file.tellg();
    file>>ch;
    cout<<" Character \" "<<ch<<"\"<<endl;

}
file.close();

}

```

Output

```

current position in file13
After open again file position is 0
Position0 Character " O"
Position1 Character " O"
Position2 Character " p"
Position3 Character " u"
Position5 Character " s"
Position6 Character " i"
Position7 Character " n"
Position8 Character " g"
Position9 Character " C"
Position11 Character " +"
Position12 Character " +"
Position13 Character " +"

Process returned 0 (0x0)   execution time : 0.136 s
Press any key to continue.

```

tellp() and seekp() function

- tellp() function returns the position of pointer then using seekp() function the pointer is shift back from n position.



Lab Exercise

```

//Program
#include <iostream>
#include<fstream>
using namespace std;
main(){
ofstream file("details.txt",ios::in);
if(!file){
    cout<<"Error";
}
else{
    cout<<file.tellp()<<endl;
}
}

```



```

    file<<"Good Programming";
    file.seekp(150);
    cout<<file.tellp()<<endl;
    file<<"Good Programming";
    file.close();
}
}

```



Task

Write a program to demonstrate working of file pointers in C++.

ignore()

This function is used when reading a file to ignore certain number of characters. You can use `seekg()` as well for this purpose just to move the pointer up in the file. However, `ignore()` function has one advantage over `seekg()` function. The prototype is of `ignore()` function is given below.

```
fstream& ignore( int, char);
```

Caution

You can specify a delimiter character in `ignore()` function whence it ignores all the characters up to the first occurrence of the specified delimiter.

Where `int` is the count of characters to be ignored and `delimiter` is the character up to which you would like to ignore as demonstrated in the following program.

```

//demonstration of ignore() function
#include <fstream.h>

void main()
{
    // Assume that the text contained in data.dat file is "Welcome
    to C++"
    ifstream myfile("data.dat");
    static char Carray[10];
    //go on ignoring all the characters in the input up to 10th
    character unless an 'm' is found
    myfile.ignore(10,'m');
    myfile.read(Carray,10);
    cout << Carray << endl; //it should display "me to C++"
    myfile.close();
}

```

getline()

This function is used to read one line at a time from an input stream until some specified criterion is met. The prototype is as follows:

```
getline(Array,Array_size,delimiter);
```

Object Oriented Programming Using C++

The stopping criterion can be either specified number of characters (Array_size) or the first occurrence of a delimiter (delimiter) else the entire line (up to newline character '\n') is read. If you wish to stop reading until one of the following happens:

1. You have read 10 characters
2. You met the letter 'm'
3. There is new line

Then the function will be called as follows:

```
getline(Carray,10,'m');
```

The use of getline() function is demonstrated in the following example.

```
//Demonstration of getline() function to read a file line-wise
#include <fstream.h>

void main()
{
    // Assume that the text contained in data.dat file is "Welcome to
    C++"

    ifstream myfile("data.dat");
    static char Carray[10];
    myfile.getline(Carray,10,'m');
    cout << Carray << endl; // the output should be "Welco"
    myfile.close();
}
```

peek()

This function returns the ASCII code of the current character from an input file stream very much like get() function, however, without moving the pointer to the next character. Therefore, any number of successive call to peek() function will return the ASCII code of same character each time. To convert the ASCII code (as returned by peek() function use char type cast) as demonstrated in the following code program.

```
//Demonstration of peek() function
#include <fstream.h>

void main()
{
    // Assume that the text contained in data.dat file is "Welcome to
    C++"

    ifstream myfile("data.dat");
    char ch;
    myfile.get(ch);
    cout << ch << endl; // the output should be 'W' and the pointer
    will move to point 'e'
    cout << char(myfile.peek()) << endl; // should display "e"
    cout << char(myfile.peek()) << endl; // should display "e" again
    cout << myfile.peek() << endl; // should display 101
    myfile.get(ch);
}
```

```
cout << ch << endl; //will display "e" again and move the
pointer to 'l'
myfile.close();
}
```

putback()

This function returns the last read character, and moves the pointer back. In other words, if you use `get()` to read a char and move the pointer to next character, then use `putback()`, it will show you the same character, but it will set the pointer to previous character, so the next time you call `get()` again, it will again show you the same character as shown in the following program.

```
//Program demonstrating use of putback() function
#include <fstream.h>
void main()
{
// Assume that the text contained in data.dat file is "Welcome to
C++"
ifstream myfile("data.dat");
char ch;
myfile.get(ch);
cout << ch << endl; //output will be 'W'
myfile.putback(ch);
cout << ch << endl; //output will again be 'W'
myfile.get(ch);
cout << ch << endl; // output will again be 'W'
myfile.close();
}
```

flush()

I/O streams are created and maintained in the RAM. Therefore, when dealing with the output file stream, the data is not saved in the file as the program enters them. A buffer in the memory holds the data until the time you close the file or the buffer is full. When you close the file the data is actually saved in the designated file on the disk. Once the data has been written to the disk the buffer becomes empty again.

In case you want to force the data be saved even though the buffer is not full without closing the file you can use the `flush()` function. A call to `flush()` function forces the data held in the buffer to be saved in the file on the disk and get the buffer empty.

10.5 Sequential Input and Output Operations

The file stream classes support a number of member functions for performing the input and output operations on files.

The `get()` and `put()` functions are capable of handling a single character at a time.

The `getline()` function lets you handle multiple characters at a time.

Sequential File Structure in C++

- C++ imposes no structure on the data stored in files, they typically are stored in an unordered format.
- If we want to set any form of order in the file data, we must do so programmatically.



Example

```
//Program
#include<iostream>
#include<fstream>
using namespace std;
int main()
{
    char fname[20], ch;
    ifstream fin;
    cout<<"Enter the name of the file: ";
    cin.get(fname, 20);
    cin.get(ch);
    fin.open(fname, ios::in);
    if(!fin)
    {
        cout<<"Error ";
    }
    while(fin)
    {
        fin.get(ch);
        cout<<ch;
    }
    fin.close();
    return 0;
}
```

Output

```
Enter the name of the file: details.txt
Helloo
Press any key to exit...

Process returned 0 (0x0)   execution time : 7.680 s
Press any key to continue.
```



Lab Exercise

```
#include<iostream>
#include<fstream>
#include<stdlib.h>
using namespace std;
int main()
{
    ofstream fout;
    char ch;
    int line=0;
    int i;
    fout.open("sample.txt", ios::app) ;
    if(!fout)
    {
        cout<<"Error"<<endl;
    }
    for(i=33; i<128; i++)
    {
        fout.put((char)(i));
    }
    fout.close();
    ifstream fin;
    fin.open("sample.txt", ios::in);
    fin.seekg(0);
    for(i=33; i<128; i++)
    {
        fin.get(ch);
        cout<<i<<" ";
        cout.put((char)(i));
        cout<<"\t";
        if(!(i%8))
        {
            cout<<"\n";
            line++;
        }
        if(line>22)
        {
            system("PAUSE");
            line = 0;
        }
    }
}
```

```

    }
    return 0;
}

```

Output

```

33 = ! 34 = " 35 = # 36 = $ 37 = % 38 = & 39 = ' 40 = (
41 = ) 42 = * 43 = + 44 = , 45 = - 46 = . 47 = / 48 = 0
49 = 1 50 = 2 51 = 3 52 = 4 53 = 5 54 = 6 55 = 7 56 = 8
57 = 9 58 = : 59 = ; 60 = < 61 = = 62 = > 63 = ? 64 = @
65 = A 66 = B 67 = C 68 = D 69 = E 70 = F 71 = G 72 = H
73 = I 74 = J 75 = K 76 = L 77 = M 78 = N 79 = O 80 = P
81 = Q 82 = R 83 = S 84 = T 85 = U 86 = V 87 = W 88 = X
89 = Y 90 = Z 91 = [ 92 = \ 93 = ] 94 = ^ 95 = _ 96 = `
97 = a 98 = b 99 = c 100 = d 101 = e 102 = f 103 = g 104 = h
105 = i 106 = j 107 = k 108 = l 109 = m 110 = n 111 = o 112 = p
113 = q 114 = r 115 = s 116 = t 117 = u 118 = v 119 = w 120 = x
121 = y 122 = z 123 = { 124 = | 125 = } 126 = ~ 127 = 
Process returned 0 (0x0)   execution time : 0.034 s
Press any key to continue.

```

10.6 Update a File

Updating a file is process of changing values in one or more records of a file.

Random Access

Now we have a file open for reading. By default, the file is open and the cursor is sitting at the beginning of the file. But now we want to move the file pointer that is the read/write location inside your file. Note that we told C++ to open the file in append mode. This means our pointer is now at the END of the file.

Once you have a file open for processing, you can navigate to different parts of the file. This is often referred to as random access of files. It really means that you aren't at the beginning or the end. Two functions are available:

istream: seekg() (or seek and get)

ostream: seekp() (seek and put)



Did you know?

Types of Files

There are three basic types of files:

Regular Stores data (text, binary, and executable).

directory Contains information used to access other files.

special Defines a FIFO (first-in, first-out) pipe file or a physical device.

All file types recognized by the system fall into one of these categories. However, the operating system uses many variations of these basic types.

Regular Files

Regular files are the most common files. Another name for regular files is ordinary files. Regular files contain data.

Text Files

Text files are regular files that contain information readable by the user. This information is stored in ASCII. You can display and print these files. The lines of a text file must not contain NUL characters, and none can exceed {LINE_MAX} bytes in length, including the new-line character.

The term text file does not prevent the inclusion of control or other nonprintable characters (other than NUL). Therefore, standard utilities that list text files as inputs or outputs are either able to process the special characters gracefully or they explicitly describe their limitations within their individual sections.

Binary Files

Binary files are regular files that contain information readable by the computer. Binary files may be executable files that instruct the system to accomplish a job. Commands and programs are stored in executable, binary files. Special compiling programs translate ASCII text into binary code.

The only difference between text and binary files is that text files have lines of less than {LINE_MAX} bytes, with no NUL characters, each terminated by a new-line character.

10.7 Different Types of Files

A file is a collection of letters, numbers, and special characters that can be used to create a programme, a database, a dissertation, a reading list, or a simple letter, among other things. You can sometimes import a file from another location, such as another computer. You'll need to create a file if you wish to insert your own text or data. You'll need to return to a file later to update its contents, whether you copied it from somewhere else or generated it yourself.

The most common file systems rely on an underlying data storage device that provides access to a set of fixed-size blocks, referred to as sectors, that are typically 512 bytes in size. The file system software is in charge of grouping these sectors into files and directories, as well as keeping track of which sectors belong to which files and which aren't. Most file systems handle data in "clusters" or "blocks," which are fixed-sized units containing a specific number of disc sectors (usually 1-64). This is the minimum amount of logical disc space that can be assigned to a file.

However, file systems need not make use of a storage device at all. A file system can be used to organize and represent access to any data, whether it be stored or dynamically generated (e.g., from a network connection).

Whether the file system has an underlying storage device or not, file systems typically have directories which associate file names with files, usually by connecting the file name to an index into a file allocation table of some sort, such as the FAT in an MS-DOS file system, or an inode in a Unix-like file system. Directory structures may be flat, or allow hierarchies where directories may contain subdirectories. In some file systems, file names are structured, with special syntax for filename extensions and version numbers. In others, file names are simple strings, and per-file metadata is stored elsewhere.

Other bookkeeping information is typically associated with each file within a file system. The length of the data contained in a file may be stored as the number of blocks allocated for the file or as an exact byte count. The time that the file was last modified may be stored as the file's timestamp. Some file systems also store the file creation time, the time it was last accessed, and the time that the file's meta-data was changed. (Note that many early PC operating systems did not keep track of file times.) Other information can include the file's device type (e.g., block, character, socket, subdirectory, etc.), its owner user-ID and group-ID, and its access permission settings (e.g., whether the file is read-only, executable, etc.).

The hierarchical file system was an early research interest of Dennis Ritchie of Unix fame; previous implementations were restricted to only a few levels, notably the IBM implementations, even of

their early databases like IMS. After the success of Unix, Ritchie extended the file system concept to every object in his later operating system developments, such as Plan 9 and Inferno.

Traditional file systems offer facilities to create, move and delete both files and directories. They lack facilities to create additional links to a directory (hard links in Unix), rename parent links (".." in Unix-like OS), and create bidirectional links to files.

Traditional file systems also offer facilities to truncate, append to, create, move, delete and in-place modify files. They do not offer facilities to prepend to or truncate from the beginning of a file, let alone arbitrary insertion into or deletion from a file. The operations provided are highly asymmetric and lack the generality to be useful in unexpected contexts. For example, interprocess pipes in Unix have to be implemented outside of the file system because the pipes concept does not offer truncation from the beginning of files.

Secure access to basic file system operations can be based on a scheme of access control lists or capabilities. Research has shown access control lists to be difficult to secure properly, which is why research operating systems tend to use capabilities. Commercial file systems still use access control lists.

A file system is a method for storing and organizing computer files and the data they contain to make it easy to find and access them. File systems may use a data storage device such as a hard disk or CD-ROM and involve maintaining the physical location of the files, they might provide access to data on a file server by acting as clients for a network protocol (e.g., NFS, SMB, or 9P clients), or they may be virtual and exist only as an access method for virtual data. More formally, a file system is a set of abstract data types that are implemented for the storage, hierarchical organization, manipulation, navigation, access, and retrieval of data.

10.8 Types of File Systems

File system types can be classified into disk file systems, network file systems and special purpose file systems.

1. **Disk file systems:** A disk file system is a file system designed for the storage of files on a data storage device, most commonly a disk drive, which might be directly or indirectly connected to the computer. Examples of disk file systems include FAT, FAT32, NTFS, HFS and HFS+, ext2, ext3, ISO 9660, ODS-5, and UDF. Some disk file systems are journaling file systems or versioning file systems.
2. **Flash file systems:** A flash file system is a file system designed for storing files on flash memory devices. These are becoming more prevalent as the number of mobile devices is increasing, and the capacity of flash memories catches up with hard drives. While a block device layer can emulate a disk drive so that a disk file system can be used on a flash device, this is suboptimal for several reasons:
 - (a) **Erasing blocks:** Flash memory blocks have to be explicitly erased before they can be written to. The time taken to erase blocks can be significant, thus it is beneficial to erase unused blocks while the device is idle.
 - (b) **Random access:** Disk file systems are optimized to avoid disk seeks whenever possible, due to the high cost of seeking. Flash memory devices impose no seek latency.
 - (c) **Wear leveling:** Flash memory devices tend to wear out when a single block is repeatedly overwritten; flash file systems are designed to spread out writes evenly.

Log-structured file systems have all the desirable properties for a flash file system. Such file systems include JFFS2 and YAFFS.

3. **Database file systems:** A new concept for file management is the concept of a database-based file system. Instead of, or in addition to, hierarchical structured management, files are identified by their characteristics, like type of file, topic, author, or similar metadata. Example: dbfs.

4. **Transactional file systems:** Each disk operation may involve changes to a number of different files and disk structures. In many cases, these changes are related, meaning that it is important that they all be executed at the same time. Take for example a bank sending another bank some money electronically. The bank's computer will "send" the transfer instruction to the other bank and also update its own records to indicate the transfer has occurred. If for some reason the computer crashes before it has had a chance to update its own records, then on reset, there will be no record of the transfer but the bank will be missing some money.

Transaction processing introduces the guarantee that at any point while it is running, a transaction can either be finished completely or reverted completely (though not necessarily both at any given point). This means that if there is a crash or power failure, after recovery, the stored state will be consistent. (Either the money will be transferred or it will not be transferred, but it won't ever go missing "in transit".)

5. **Network file systems:** A network file system is a file system that acts as a client for a remote file access protocol, providing access to files on a server. Examples of network file systems include clients for the NFS, SMB protocols, and file-system-like clients for FTP and WebDAV.
6. **Special purpose file systems:** A special purpose file system is basically any file system that is not a disk file system or network file system. This includes systems where the files are arranged dynamically by software, intended for such purposes as communication between computer processes or temporary file space.

Special purpose file systems are most commonly used by file-centric operating systems such as Unix. Examples include the procfs (/proc) file system used by some Unix variants, which grants access to information about processes and other operating system features.

10.9 Binary Files

In binary files, to input and output data with the extraction and insertion operators (<< and >>) and functions like `getline` is not efficient, since we do not need to format any data, and data may not use the separation codes used by text files to separate elements (like space, newline, etc...). File streams include two member functions specifically designed to input and output binary data sequentially: `write` and `read`. The first one (`write`) is a member function of `ostream` inherited by `ofstream`. And `read` is a member function of `istream` that is inherited by `ifstream`. Objects of class `fstream` have both members. Their prototypes are:

```
write ( memory_block, size );
```

```
read ( memory_block, size );
```

Where `memory_block` is of type "pointer to char" (`char*`), and represents the address of an array of bytes where the read data elements are stored or from where the data elements to be written are taken. The `size` parameter is an integer value that specifies the number of characters to be read or written from/to the memory block.



Example

```
// reading a complete binary file
#include <iostream>
#include <fstream>
using namespace std;
ifstream::pos_type size;
char * memblock;
int main () {
```

Object Oriented Programming Using C++

```
ifstream file ("example.bin", ios::in | ios::binary | ios::ate);
if (file.is_open())
{
    size = file.tellg();
    memblock = new char [size];
    file.seekg (0, ios::beg);
    file.read (memblock, size);
    file.close();
    cout << "the complete file content is in memory";
    delete[] memblock;
}
else cout << "Unable to open file";
return 0;
} the complete file content is in memory
```

10.10 Command Line Argument in C++

Any input value is passed through command prompt at the time of running of program is known as command line argument.

- They facilitate the use of your program in batch files
- They give a professional appearance to your program

A command-line argument is the information that follows the name of the program on the command line of the operating system.

C++ defines two built-in parameters to main()

- They receive the command line arguments
- Their names are argc and argv

The names of the parameters are arbitrary. However, argc and argv have been used by convention for several years.

```
int main( int argc ,char *argv[])
```

- argc is an integer Holds the number of arguments on the command line Since the name of the program always corresponds to the first argument, it is always at least 1

argv is a pointer to an array of character pointers.

Each character pointer in the argv array corresponds a string containing a command-line argument

argv[0] points the name of the program, argv[1] points to the first argument, argv[2] points to the second argument and so in



Note: Each command-line argument is a string

If you want to pass numerical information to your program, your program should convert the corresponding argument into its numerical equivalent



Note: Each command-line argument must be separated by spaces or tabs

Commas, semicolons, and the like are not valid argument separators

Passing numeric command-line arguments

All command-line arguments are passed to the program as strings

- Program should convert them into their proper internal format

C++ supports standard library functions

- **atof()** : converts a string to a double and returns the result
- **atoi()** : converts a string to a int and returns the result
- **atol()** : converts a string to a long int and returns the result

Properties of Command Line Arguments

- They are passed to main() function.
- They are parameters/arguments supplied to the program when it is invoked.
- They are used to control program from outside instead of hard coding those values inside the code.
- argv[argc] is a NULL pointer.
- argv[0] holds the name of the program.
- argv[1] points to the first command line argument and argv[n] points last argument.

Operating System parses command line arguments

An Operating System (OS) is an interface between a computer user and computer hardware.

Operating System

- Command Line Interface
- Graphical User Interface

Command Line Interface

A command-line interface (CLI) processes commands to a computer program in the form of lines of text.

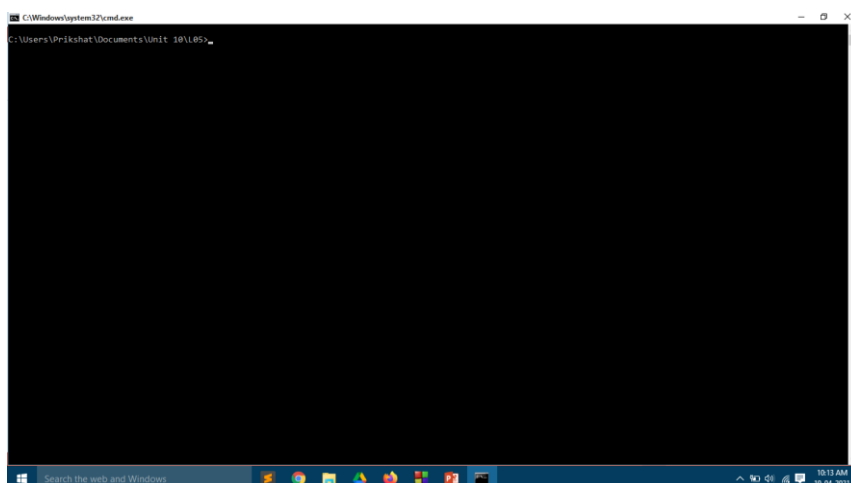


Example

LINUX

UNIX

DOS



Graphical user Interface

A GUI (graphical user interface) is a system of interactive visual components for computer software.

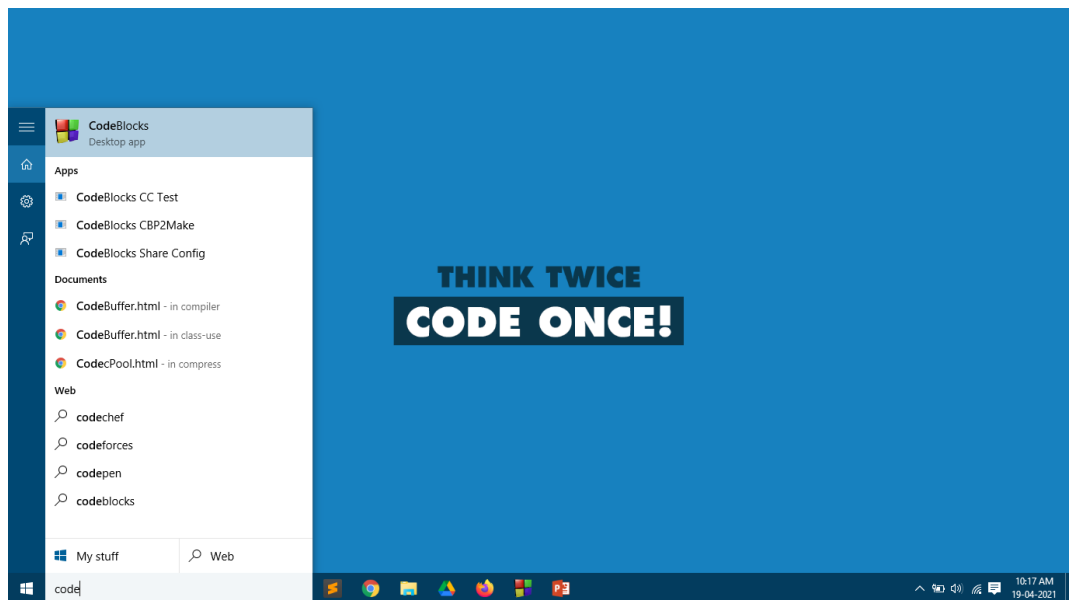


Example

WINDOWS OS

MAC OS

UBUNTU (LINUX)



Lab Exercise

```
#include<iostream>
using namespace std;
int main(int argc, char* argv[])
{
    int i;
    cout<<"Total number of arguments: "<<argc;
    for(i=0;i< argc;i++)
    {
        cout<<endl<< i<<"argument: "<<argv[i];
    }
    return 0;
}
Output
```

No argument passed

```
Total number of arguments: 1
0argument: C:\Users\Prikshat\Desktop\a.exe
Process returned 0 (0x0)   execution time : 0.095 s
Press any key to continue.
```

Command line arguments passed

```
C:\Users\Prikshat\Desktop>a.exe 12 21 35 25 36
Total number of arguments: 6
0argument: a.exe
1argument: 12
2argument: 21
3argument: 35
4argument: 25
5argument: 36
C:\Users\Prikshat\Desktop>
```

Summary

- Fields in C++ are interpreted as a sequence of or stream of bytes stored on some storage media'. Member functions of these or base classes are used to perform I/O operations.
- The read() and write() functions work in binary mode. The ifstream class is used for input, ofstream for output and istream for both input and output.
- A data of a file is stored in the form of readable and printable characters then the file is known as text file. A file contains non-readable characters in binary code then the file is called binary file.
- The function get() read and write data respectively. The read() and write() function read and write block of binary data. The close() function close the stream.
- C++ treats each source of input and output uniformly. The abstraction of a data source and data sink is what is termed as stream. A stream is a data abstraction for input/output of data to and fro the program.
- C++ library provides prefabricated classes for data streaming activities. In C++, the file stream classes are designed with the idea that a file should simply be viewed as a stream or array or sequence of bytes.

Keywords

Command Line Parameters: The main functions may be defined not to have any parameter. In some cases, though, the program is provided with some input values at the line of execution. These values are known as command line parameter.

File: A storage unit that contains data. A file can be stored either on tape or disk.

Input Stream: The stream that supplies data to the program is known as input stream.

Output Stream: The stream that receives data from the program is known as output stream.

Stream: A stream is a general name given to a flow of data.

Self Assessment

1. Which header file is required to use file I/O operations?
 - A. <ifstream>
 - B. <ostream>
 - C. <fstream>
 - D. <iostream>

2. By default, all the files in C++ are opened in _____ mode.
 - A. Text
 - B. Binary
 - C. ISCI
 - D. VTC

3. Which among following is used to open a file in binary mode ?
 - A. ios::app
 - B. ios::out
 - C. ios::in
 - D. ios::binary

4. Which function is used in C++ to get the current position of file pointer in a file?
 - A. tell_p()
 - B. get_pos()
 - C. get_p()
 - D. tell_pos()

5. The tellg() function is used with input streams and returns the current "get" position of the pointer in the stream.
 - A. True
 - B. False

6. By default reading pointer is set at the end when file open in ios::app mode.
 - A. True
 - B. False

7. The get() and put() functions are capable of handling a single character at a time.
 - A. True
 - B. False

8. Which of the following is the correct syntax to read the single character to console in the C++ language?
 - A. Read ch()
 - B. Getline vh()

- C. get(ch)
 - D. Scanf(ch)
9. Which of the following is the correct syntax to read the single character to console in the C++ language?
- A. Print ch()
 - B. Printtline vh()
 - C. put(ch)
 - D. Scanf(ch)
10. Updating of the file means
- A. adding new data
 - B. deleting any existing data
 - C. changing the existing record items
 - D. all of above
11. For a direct access file :
- A. there are restrictions on the order of reading and writing
 - B. there are no restrictions on the order of reading and writing
 - C. access is restricted permission wise
 - D. access is not restricted permission wise
12. The command line arguments are handled using?
- A. void()
 - B. main()
 - C. header files
 - D. macros
13. argc refers to the?
- A. number of arguments passed
 - B. a pointer array
 - C. Both number of argument passed and pointer array
 - D. None of the above
14. What will be output for the following code?
- ```
#include <stdio.h>

int main(int argc, char *argv[])
{
 printf("%d", argc);
 return 0;
}
```
- A. 0
  - B. 1
  - C. error
  - D. Depends on the compiler

15. What does the second parameter of the main function represent?
- A. Number of command line arguments
  - B. List of command line arguments
  - C. Dictionary of command line arguments
  - D. Stack of command line arguments

### **Review Questions**

1. How is C++ able to treat all the input and output operation uniformly?
2. What do you mean by file pointers? Explain in detail.
3. Write a note on read() and write() functions.
4. What is a file? Explain different types of files.
5. How can you achieve random access in C++?
6. Write a program in C++ to create a file.
7. Write a program in C++ that creates a text file, which is an exact copy of a given file.

### **Answers for Self Assessment**

- |       |       |       |       |       |
|-------|-------|-------|-------|-------|
| 1. C  | 2. A  | 3. D  | 4. A  | 5. A  |
| 6. B  | 7. A  | 8. C  | 9. C  | 10. D |
| 11. B | 12. B | 13. A | 14. B | 15. B |



### **Further Readings**

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.



### **Web Links**

<https://study.com/academy/lesson/practical-application-for-c-plus-plus-programming-working-with-files.html>

<https://www.codecademy.com/learn/learn-c-plus-plus>

<https://www.guru99.com/cpp-file-read-write-open.html>